

Telemetria UART

Deteccao de erros

Roteiro para preparar a entrega principal e defender o trabalho

Principal: teoria, especificacao, exemplos manuais, demonstracao no computador e estrutura do relatorio.

Extra separado: montagem e demonstracao fisica com tres ESP32.

Base: curso proprio de 21 dias (47 paginas) e enunciado da atividade (2 paginas), fornecidos pelo estudante.

Grupo de tres estudantes; defesa e nota individuais. Use este guia para estudar e executar, e escreva a analise com suas palavras. Este PDF nao substitui um relatorio final com evidencias autorais.

Repositorio: github.com/enzogomes2006/TCD_RUIDO

Como navegar

1. Roteiro de defesa de 20 minutos e demonstracoes teoricas.
2. Especificacao proposta do protocolo.
3. Estrutura editavel do relatorio.
4. Matriz de requisitos e evidencias.
5. Extra: bancada, piloto e coleta real.
6. Montagem BC547 e piloto de vinte quadros.
7. Registro de validacao do codigo.

Escopo: a parte fisica e extra segundo a orientacao do estudante. O PDF original pede ruido real, medicoes e demo ao vivo. A matriz registra as evidencias que dependeriam desse escopo experimental.

Atualizado em 7 de outubro de 2026 | BC547 e validacao do codigo

Roteiro da entrega principal e defesa - 20 minutos

Use como guia de estudo, sem decorar frases para atribuir autoria a um texto que a equipe não escreveu. O curso é a base; a equipe deve explicar o raciocínio com suas palavras. A montagem física fica no arquivo `extra_bancada.md`.

Preparação em ordem

1. Dias 1..7 do curso: revisar binário, hex, máscaras e UART. Produto: desenhar 80 e 00 em bits e calcular 8,68 us/bit.
2. Dias 8..14: revisar quadro, SEQ, RX, soma, CRC e timeout. Produto: especificação revisada e exemplo manual resolvido.
3. Dias 15..20: estudar o modelo de ruído e as métricas. Para a entrega principal, executar o modelo no PC e identificar claramente que os dados são simulados. A bancada é uma extensão separada.
4. Dia 21: ensaiar a defesa com tempo, perguntas e troca de papéis. Produto: cada integrante explica uma parte que não implementou.
5. Escrever o relatório a partir das evidências existentes; conferir a matriz de requisitos; exportar PDF; testar o link GitHub sem depender da conta do autor.

Defesa cronometrada

Tempo	Responsavel inicial	Mostrar	Explicar
0..2 min	A	objetivo e escopo	medir dados errados aceitos; teoria/simulacao e extra separados
2..5 min	A	desenho dos oito bytes e UART	START, LEN, SEQ, payload e CHECK; uma tecnica por vez
5..8 min	B	fluxo RX e dois exemplos	decide sem conhecer verdade; residuo, descarte, referencia posterior
8..11 min	C	colisao manual	dois bits 1->0, soma diminui 256, payload errado aceito
11..14 min	C	CRC e desenho experimental	polinomio, parametros, 7000 quadros no plano, variaveis constantes
14..16 min	B	CSV/tabelas rotulados	classes exclusivas, taxa e limites do modelo
16..18 min	os tres	previsoes de 10 quadros	expectativa estatistica antes da execucao; comparar depois
18..20 min	os tres	conclusao e perguntas	responder com evidencias e reconhecer o que falta medir

Essa e uma distribuicao sugerida. Cada aluno estuda todas as partes porque a nota e individual. Se o professor exigir demo real dentro dos 20 minutos, o grupo precisa ajustar o roteiro; nao acrescente bancada obrigatoria silenciosamente ao escopo teorico.

Demonstração teorica 1: quadro correto

Escrever no papel:

```
START LEN SEQ_H SEQ_L P0 P1 CHECK
AA 55 05 00 01 80 80 FA
```

Soma protegida: $5+0+1+128+128=262$. $262 \bmod 256=6$. $CHECK=-6 \bmod 256=250=FA$.

RX processa $5+0+1+128+128+250=512$; $512 \bmod 256=0$. Aceita. Apenas depois compara payload com a referencia: 80 80 == 80 80, portanto OK.

Demonstração teórica 2: um erro detectado

Apagar apenas o MSB de P0: AA 55 05 00 01 00 80 FA.

RX soma $5+0+1+0+128+250=384$; residuo $128=80$, diferente de zero. Descarta. Classe CORROMPIDO mesmo que apenas o CHECK tivesse mudado; a classe é dada pela falha de checagem.

Demonstração teórica 3: quebrar SUM8

Apagar os MSBs dos dois bytes: AA 55 05 00 01 00 00 FA.

P0: 10000000 -> 00000000. P1: 10000000 -> 00000000.

Soma recebida: $5+0+1+0+0+250=256$. Residuo zero. RX aceita. A referencia posterior revela que 00 00 != 80 80: NAO_DETECTADO.

Delta total= $-128-128=-256$, que equivale a zero modulo 256. Isso ilustra um padrao compativel com LOW. Nao usar como unica explicacao o exemplo $10+20 \rightarrow 11+1F$, porque ele exige uma mudanca de 0 para 1.

Este caso é matematico e reproduzido pelos testes, sem provar uma ocorrencia em fio. O enunciado PDF pede caso real: essa evidencia fica pendente se não houver dispensa ou bancada.

CRC: explicar e calcular

Usar CRC-8/SMBUS: largura 8, poly 07, init 00, sem reflexao, xor final 00. Para cada byte: XOR com o registrador; repetir oito shifts, com XOR 07 quando o bit superior era 1. Mostrar no quadro um passo e deixar o calculo completo disponivel.

No mesmo corpo 05 00 01 80 80, obter o CHECK pelo codigo e verificar os seis bytes incluindo CHECK. Repetir apos apagar os dois MSBs. O teste confirma que esse padrao é rejeitado pelo CRC escolhido.

CRC de grau 8 detecta todo erro não nulo cuja extensao matematica é no maximo oito bits, desde que o codigo seja avaliado sobre um bloco devidamente montado. Isso não garante contra perda de bytes, falso alinhamento, todo padrao mais longo ou qualquer protocolo. Zero erros vistos não significa impossibilidade de erros.

Demonstracao no PC

Abrir tres janelas/arquivos: especificacao, terminal e CSV. Rodar os comandos do README. Explicar que o modelo forca LOW em memoria, mantem o framing e usa apenas payload/check. Por isso não valida PERDIDO, timeout eletrico ou recuperacao real.

Abrir sum8_mod02.csv, filtrar NAO_DETECTADO e conferir TX hex, RX hex, bits tentados e bits alterados. Calcular manualmente o caso encontrado. Se o grupo desejar mostrar exatamente o exemplo anterior, usar o teste test_low_only_collision como exemplo construido e rotula-lo assim.

Gerar duas tabelas apenas para demonstracao. Não transferir esses CSVs para data/reais/.

Predicao dos proximos 10

Para cada classe, $p=\text{count}/1000$; expectativa de dez quadros $E=10*p$. Se a tabela de estudo tiver 200 OK, 750 CORROMPIDOS, 0 PERDIDOS e 50 ND, expectativas são 2;7,5;0;0,5. É um exemplo hipotetico, não resultado do grupo.

Cada aluno registra uma previsao inteira somando 10 e justifica a variabilidade. A expectativa decimal não é uma promessa de contagem exata. A referencia do modo escolhido precisa estar disponivel antes de prever; se o professor não revela nenhum indicio do modo, só é possivel uma previsao condicional ou agregada, e isso deve ser esclarecido.

Na demonstração teórica, compare com outro bloco simulado de dez quadros. Na bancada extra, compare com o que RX registrou. Grave previsões antes dos resultados, evitando retrospectiva.

Perguntas para os três

- Por que UART 8N1 tem dez bits por byte? Qual bit de dados sai primeiro?
- Por que SEQ não substitui CHECK? O que acontece no wrap?
- Por que referência não pode ajudar a aceitar? Como evitar vazamento da verdade?
- Qual diferença entre CORRUMPIDO e PERDIDO? Como evitar dupla contagem?
- Por que o exemplo de dois MSBs passa na soma? Qual o resíduo exato?
- Quais parâmetros identificam o CRC? Qual é a ordem matemática versus UART?
- Por que um pulso sobre bit zero não muda o dado?
- O que a simulação deixou de modelar? Que conclusões não podem ser tiradas dela?
- O timeout foi medido ou proposto? Como o grupo o justificaria?
- Por que a taxa não inclui OK nem PERDIDO? O que fazer com denominador zero?
- Qual contribuição individual você consegue demonstrar no histórico?

Checklist de envio

Relatório autoral PDF; link GitHub e versão; especificação coerente com código; duas tabelas com origem; fontes; contribuições; exemplos manuais; limites; tudo que foi medido acompanhado de registro. Não declarar cumprimento experimental quando só há simulação.

Especificacao proposta v0.1 - revisar e congelar antes da bancada

Enlace e formato

UART2, 115200 baud, 8N1, GND comum. Sem retransmissao. Uma tecnica por rodada.

Checksum SUM8 e CRC-8 usam exatamente o mesmo campo de 1 byte; nenhuma rodada usa ambos.

Offset	Campo	Bytes	Regra
0..1	START	2	AA 55
2	LEN	1	05: tamanho de SEQ+PAYLOAD+CHECK
3..4	SEQ	2	uint16 big endian; comeca em 0 e volta de 65535 para 0
5..6	PAYLOAD	2	P0 e P1; RX nao conhece o gerador
7	CHECK	1	SUM8 ou CRC-8/SMBUS

Quadro: 8 bytes. Payload: 2 bytes. Overhead: 6 bytes = 300% do payload, ou 75% do quadro. Especifique o denominador ao usar percentual. Referencia e USB nao entram no overhead.

Escolhemos comprimento para delimitar por contagem, evitando tratar um marcador de fim dentro do payload como terminador. LEN e fixo e deve ser 05. START pode aparecer nos dados: durante quadro reconhecido ele nao reinicia o parser; fora do quadro ele e procurado novamente.

Protegidos: LEN, SEQ alto, SEQ baixo, P0 e P1. O RX tambem processa CHECK na verificacao. START nao e protegido: alteracao nele pode causar perda de sincronismo.

SUM8

$CHECK = -(LEN + SEQ_H + SEQ_L + P0 + P1) \bmod 256$.

RX: soma dos seis bytes, incluindo CHECK, modulo 256. Residuo esperado: 0.

Essa convencao difere do exemplo do curso que envia a soma positiva e depois compara. Aqui a convencao de residuo satisfaz explicitamente o enunciado.

CRC-8/SMBUS

width=8; poly=0x07 (x^8+x^2+x+1); init=0x00; refin=false; refout=false; xorout=0x00.

Processamento matematico MSB primeiro por byte. UART envia cada byte LSB primeiro: sao ordens distintas e isso deve ser explicado.

CHECK = CRC dos cinco bytes protegidos. RX processa os seis bytes. Residuo esperado: 00. Vetor de teste ASCII 123456789: F4.

Temporizacao proposta

Tbit = $1/115200 = 8,6806 \text{ us}$. Byte 8N1 = 10 bits = 86,8056 us.

8 bytes exigem 694,44 us de transmissao efetiva. Depois dos primeiros cinco bytes o TX reserva uma pausa nominal de 2 ms para armar o prototipo do injetor. A duracao real pode ser maior pelo escalonamento.

Período nominal entre quadros: 100 ms. A referência vem após o quadro e uma pausa nominal de 1 ms.

Essas pausas são constantes nas duas técnicas e precisam aparecer na medição; não são bytes de overhead.

Timeout parcial inicial: 10 ms desde o último byte consumido, em FRAME ou DEBUG. Timeout de ausência inicial: 250 ms desde a última checagem concluída, ou desde inicialização/reset.

São valores propostos, não medidos. Confirme no piloto os intervalos entre bytes, a pausa de armamento e o período entre quadros. Justifique a margem e ajuste as constantes com os dados. Logs de consumo USB/UART não substituem captura dos pulsos do injetor.

Decisão, referência e resincronização

1. Procure AA 55. Leia LEN e, se igual a 05, conte os cinco bytes seguintes.
2. Ao terminar, calcule o resíduo antes de procurar/ler a referência. Aceite apenas se zero; se falhar, descarte o payload da aplicação.
3. Salve bytes e decisão para auditoria; volte a procurar marcadores a partir do byte imediatamente posterior ao último byte consumido. A especificação não faz busca retroativa no quadro rejeitado.
4. LEN inválido ou timeout abandona a montagem e volta a SEEK. Pode haver falso alinhamento; os testes de recuperação precisam mostrar sua duração.
5. Referência separada: D3 3D SEQ_H SEQ_L P0 P1. Tem seis bytes, não recebe código de detecção e não altera a decisão. Pode associar a sequência verdadeira ao registro somente depois da decisão.
6. Sem checagem concluída e com referência identificada: PERDIDO. Com checagem falha: CORROMPIDO. Com checagem zero e payload igual: OK. Com checagem zero e payload diferente: NAO_DETECTADO.
7. Referência ausente: SEM_REFERENCIA e bloqueio de fechamento da tabela. Não chamar automaticamente de OK ou PERDIDO.

Sequência sem dupla contagem

Para quadros aceitos, $\text{delta} = (\text{SEQ_recebida} - \text{SEQ_esperada}) \bmod 65536$. $\text{Delta}=0$ é normal. 1..32767 indica lacuna observada; depois espera $\text{SEQ_recebida}+1 \bmod 65536$. 32768..65535 indica repetido/antigo/ambiguidade; não avança expected nem infere milhares de perdas.

A primeira sequência aceita ancora expected. O RX não confia em SEQ de um quadro rejeitado. Um SEQ corrompido que passou pela checagem pode enganar o rastreamento; isso é uma limitação e deve ser auditado.

Lacuna observada não é contagem final de PERDIDO: pode incluir quadros anteriormente CORROMPIDOS. O timeout registra evento provisório, nunca acrescenta a mesma perda a cada passagem do loop. Para fechar uma rodada, cruze TX e RX por sequência verdadeira e atribua uma única classe a cada quadro transmitido. Se quadro e referência desaparecerem, o log TX é necessário para a reconciliação; os sketches não automatizam toda essa reconciliação.

Domínio de ruído do protótipo extra

Uniforme sobre os 24 bits de dados dos offsets 5,6,7. Modo 1: um bit; modo 2: duas posições distintas; modo 3: byte escolhido e rajada de 3..8 bits dentro dele. Tenta forçar LOW, nunca elevar 0 para 1. Portanto comprimento tentado não é número de bits efetivamente alterados.

Escolha aleatoria restrita a payload/check e uma simplificacao declarada. Ela nao experimenta corrupcao do marcador, LEN ou SEQ. Para avaliar perda/ressincronizacao, faca testes separados de truncamento/perda de bytes; ampliar o injetor para todo o quadro exige outro mecanismo de sincronizacao e revisao do codigo.

Detecção de erro em telemetria UART com ESP32

Modelo de estrutura, não relatório final. Escrever os parágrafos a partir do estudo e das evidências da equipe. Exportar em PDF somente após completar e revisar.

Instituição: [preencher]. Disciplina: [preencher]. Professor: [preencher]. Data: [preencher]. Integrantes: [três nomes].

Código e documentação: https://github.com/enzogomes2006/TCD_RUIDO

Versão/commit analisado: [preencher].

1. Objetivo e escopo autorizado

Escrevam qual pergunta o trabalho responde: quando dados corrompidos passam pela checagem? Registrem se a avaliação principal é teórica e a bancada é extra, conforme orientação atual do professor. Identifiquem quais evidências são teóricas, simuladas ou medidas. Não anunciem 'ruído real' como resultado se não houve coleta real.

2. Fundamentação

Expliquem bit, byte, hexadecimal, UART 8N1, 8,68 us/bit, checksum modular, resíduo e CRC com parâmetros. Mostrem cálculo manual. Usem o curso como apoio e citem as fontes realmente estudadas.

3. Especificação

Incluam a tabela byte a byte, endianness, campos protegidos, critério de aceite/descartado, referência fora da decisão, uma técnica por rodada e ausência de retransmissão. Justifiquem LEN, política de sequência, wrap, repetidos e timeouts. Diagramem a máquina de estados.

4. Método e reprodutibilidade

Expliquem versões, placa-alvo, parâmetros de ruído e tamanho da amostra. Se usarem simulação, escrevam 'forçar bits para zero em memória, sem modelar camada elétrica'. Se usarem bancada extra, incluam circuito, componentes, posições, modo, timer, validação e CSVs. Não misturem os conjuntos de dados.

5. Resultados e limites

Insiram uma tabela por família. Cada linha representa uma rodada; quatro classes somam 1000. Taxa = $ND/(CORROMPIDO+ND)$; N/A para denominador zero. Se os dados forem simulados, identifiquem isso no título e na coluna origem.

Não preencher tempo TX, tempo RX, timeout justificado ou recuperação física com tempos medidos no PC. Calculem overhead de 6 bytes sem referência e expliquem o denominador do percentual.

Discutam tentativas sem mudança, erros no campo CHECK, perdas que não entraram no denominador e limite de observar zero casos numa amostra.

6. Estudo de caso byte a byte

Para o exemplo teórico: enviado AA 55 05 00 01 80 80 FA; recebido AA 55 05 00 01 00 00 FA. Expliquem os dois MSBs e a soma $0x100$ módulo 256 igual a zero.

Se o professor exigir caso real, substituam/adicionem o registro real com sequência, TX, RX, log do injetor, timestamp, mudanças confirmadas e cálculo feito pela equipe. Exemplo teórico não prova medição.

7. Predicao e defesa

Descrevam como obter $p_{\text{classe}} = \text{contagem}/1000$ e $E_{\text{classe}} = 10 * p_{\text{classe}}$. Guardem previsao individual antes da rodada de 10 quadros, justifiquem incerteza e comparem apos a execucao.

8. Conclusao autoral

Respondam a pergunta inicial com as evidencias que existem. Nao escrevam que CRC elimina todo erro. Indiquem limites, mudancas que a equipe faria e quais afirmacoes nao puderam ser verificadas.

9. Contribuicoes e referencias

Documentem contribuicoes individuais e apoio de ferramentas conforme a regra da disciplina. Incluam curso do autor, enunciado, documentacao Espressif e outras fontes que de fato usaram.

10. Anexos

Link/commit de codigo, especificacao, instrucoes de execucao, tabelas, CSVs e, se houver, capturas reais. Nunca apresentar CSV sintetico como bancada.

Matriz de atendimento e evidencias

O usuario definiu a demonstracao fisica como EXTRA. Esta matriz preserva a distincao entre essa orientacao e o PDF, que solicita ruido real. 'Previsto' nao significa 'comprovado'.

Requisito	Onde preparar	Evidencia para fechar	Situacao
Relatorio PDF bem explicado e link GitHub	modelo + guia + README	PDF autoral com link funcionando	modelo pronto; texto final pendente
Defesa individual em grupo de 3	roteiro + contribuicoes	cada aluno explica todo o sistema	ensaio pendente
Especificacao antes do codigo	especificacao.md	equipe revisa e registra versao	proposta v0.1
START, SEQ, payload ≥ 2 , uma tecnica, LEN/fim	protocolo de 8 bytes	desenho e exemplo	proposto e testado no modelo
RX calcula residuo incluindo CHECK	biblioteca + scripts + RX	exemplos de aceite e descartado	modelo testado; placa pendente
Descarte sem entregar dados/retransmitir	especificacao + RX	codigo e log	descrito; validacao pendente
Lacunas, wrap, repetido, fora de ordem	especificacao + testes	teste de 65535->0, lacuna e repeticao	modelo testado; reconciliacao real pendente
Timeout medido e recuperacao	extra + TX/RX	captura ou log real, justificativa, tempo em ms	valores iniciais; nao medidos
UART2, 115200, GND, tres ESP32	extra + firmware	montagem e captura	extra por orientacao do usuario
NPN + resistor e LOW assimetrico	extra	foto/circuito e log alteracao real	extra; estrutura iniciada, eletrica pendente
Modos 0..3 via Monitor Serial	noise.ino	troca sem regravar	compilado; teste na placa pendente
Pulso menor que byte; rajada no mesmo byte	noise.ino + especificacao	analizador logico confirma posicao/duracao	temporizacao pendente
Timer adequado; sem delayMicroseconds	noise.ino	GPTimer e captura temporal	implementacao proposta
Log ruido: seq/modo/bits/tempo/mudou	noise.ino	log cruzado com RX e TX	nao coletado
Referencia depois da decisao e sem ruido	TX/RX + protocolo	captura e revisao fluxo	modelo correto; bancada pendente

Requisito	Onde preparar	Evidencia para fechar	Situacao
1000/modo: rodada1 0..3; rodada2 1..3	simulacao + extra	7000 quadros reais se exigidos	simulacao disponivel; reais ausentes
Duas tabelas, quatro classes somam 1000	tabelas + analisador	CSV reconciliado sem pendencias	modelos vazios
Taxa ND/(CORROMPIDO+ND)	roteiro + analisador	denominador e N/A se zero	implementada
Tempo TX/RX, overhead, recuperacao	TX/RX + protocolo	medidas reais e metodo	overhead teorico 6 bytes; tempos pendentes
Um erro nao detectado REAL byte a byte	roteiro + extra	TX hex + RX hex + log ruido + calculo	exemplo teorico pronto; real ausente
Predicao individual dos proximos 10	roteiro	previsoes anteriores e resultados	ensaio pendente
Quebrar o proprio protocolo	roteiro + testes	padrao + algebra + evidencia exigida	colisao teorica SUM8 testada
tx.ino, rx.ino, noise.ino compilaveis	firmware + compilar.ps1	compilacao com versao/placa registradas	tres compilados: ESP32 classico, core 3.3.12; ver validacao_codigo.md
Autoria e dominio individual	contribuicoes + README	registro honesto e defesa pessoal	a construir pela equipe

Nota editorial: a linha 'OK' no PDF contem 'checagem indicou erro', embora diga 'aceitou'. Interpretacao consistente com o restante: OK significa checagem passou e payload correto. Explicitem isso no relatorio.

EXTRA - demonstracao fisica e coleta com tres ESP32

Esta extensao esta separada da entrega principal a pedido do estudante. Se o professor mantiver o enunciado PDF integral, parte dessas evidencias passa a ser necessaria para cumprir o trabalho; a montagem ainda nao foi realizada aqui.

Materiais e circuito proposto para revisar

Atualizacao de 7/10/2026: o transistor disponivel e um BC547. O roteiro especifico das placas, pinagem e piloto esta em montagem_bc547.md (montagem_bc547.md). O BC547 substitui o NPN proposto, respeitando sua propria pinagem; os pulsos precisam ser medidos com o componente real.

Tres ESP32 classicos com UART2 e GPIO16/17 livres; tres cabos USB; GND comum; transistor NPN BC547 com pinagem conferida; resistor serie da linha de 1 kohm; resistor de base proposto de 1 kohm; resistor base-emissor proposto de 10 kohm para mante-lo desligado; analisador logico/osciloscopio para validar pulsos.

Confirme a pinagem real do transistor pelo fabricante e a variante da placa. Nao conectar uma saida GPIO diretamente a outra saida para disputar niveis. O resistor serie limita a corrente quando NPN conduz; resistor de base tem funcao distinta.

```
TX GPIO17 --- ponto A --- Rserie 1k --- ponto B --- RX GPIO16
      |                               |
      +-- NOISE GPIO16               +-- NOISE GPIO26
      +-- NOISE GPIO25               +-- coletor NPN
                                      emissor -- GND comum

NOISE GPIO27 -- Rbase 1k -- base NPN
                |
                Rpull 10k para GND
```

Ponto A e observado antes da perturbacao; ponto B depois. GPIO25 captura a primeira borda do corpo apos a pausa de armamento. GPIO16/26 permitem comparar os bytes observados. TX GPIO16 e RX GPIO17 podem ser cruzados para a ligacao de retorno pedida no desenho da disciplina, mas o protocolo nao transmite respostas nem retransmissoes.

Valide HIGH/LOW no ponto B e largura dos pulsos com a montagem real antes de coletar amostras. A documentacao nao comprova comportamento eletrico so por descrever o circuito.

Preparacao e limites importantes

Compilar todos os sketches com a mesma biblioteca e registrar a versao Arduino-ESP32, placa/FQBN e commit. Os tres foram compilados em 7/10/2026 com Arduino-ESP32 3.3.12 e esp32:esp32:esp32; consulte validacao_codigo.md (validacao_codigo.md). tools/compiler.ps1 reproduz a compilacao, sem gravar placas.

O injetor usa GPTimer a 1 MHz, ISR de borda e eventos para ligar/desligar o NPN. Nao usa delayMicroseconds. Ter timer nao garante posicionamento: latencia de ISR, arredondamento e leitura UART devem ser medidos. TIMING_INVALIDO ou SEM_PULSO invalida a tentativa e exige corrigir o piloto.

O instante registrado e o inicio do corpo observado pelo injetor, nao o instante exato de cada borda eletrica. Use a posicao planejada junto com captura para comprovar cada pulso.

O sorteio e restrito aos 24 bits de dados dos bytes de payload/CHECK. Nao testa marcador/LEN/SEQ, start/stop ou perdas por enquadramento. Se a avaliacao exigir sorteio em todo o quadro, este prototipo precisa ser ampliado; nao declarar equivalencia.

Modo 3 une bits contíguos num pulso de aproximadamente 3..8 tempos de bit dentro de um byte. A extensão real de bits alterados pode ser menor porque havia zeros no intervalo.

Piloto antes das 7000 amostras

1. Sem NPN ativo, confirmar TX/RX em modo 0 e pelo menos vinte quadros corretamente enquadrados.
2. Confirmar que o RX decide antes de acessar a referência; verificar que a referência segue intacta.
3. Medir intervalo entre quadros e entre bytes, incluindo a pausa de 2 ms. Ajustar timeout parcial e de ausência, justificar a margem com dados.
4. Testar modo 1 sobre bit conhecido igual a 1 e sobre bit igual a 0. Conferir pulsos, antes/depois e RX; GPIO LOW deve desligar o NPN.
5. Testar modos 2 e 3 sem acertar start/stop UART ou referência. Confirmar as posições do log com analisador.
6. Testar truncamento separado para justificar timeout e medir recuperação do primeiro quadro válido. Esses testes não substituem os modos 1..3.
7. Testar sequência repetida, fora de ordem e wrap em uma rodada de validação separada.

Coleta de uma rodada

Configurar RX: r, s ou c, e o modo 0..3 como metadado. Configurar injetor: modo correspondente. Configurar TX: s ou c; primeiro enviar t para o piloto de 20 quadros, e depois g para 1000 quadros. Não trocar técnica/modo no meio da rodada.

Capturar os três monitores simultaneamente por ao menos 110 segundos. O TX fica parado depois dos mil; reiniciar com g abre nova rodada a partir de seq=0. Separe arquivos por técnica/modo/rodada.

```
python -m pip install -r requirements.txt
# Exemplo de portas; substituir pelo que o computador identificar.
python scripts/capturar.py --port COM5 --out data/reaais/sum8_mod01_rx.csv --seconds 120
--command r --command s --command 1
```

A captura abre a porta e pode reiniciar a placa; configure todas e aguarde estabilizar antes de iniciar TX. O script não automatiza sincronização dos três monitores nem junta suas linhas.

Linhas # são eventos: removê-las da tabela classificada, preservando o log bruto. Reconciliar cada quadro do TX com RX e ruído. Quadros corrompidos não entram novamente como perdidos por lacuna. Sem referência ou captura incompleta: resolver a pendência; não inventar classe.

O analisador espera CSV final com cabeçalho e exatamente 1000 sequências distintas. Se tudo estiver resolvido, agrega as quatro classes e calcula taxa. A reconciliação completa de perda de quadro+referência e manual neste protótipo.

Plano das duas rodadas

SUM8: modos 0,1,2,3; 1000 por modo. CRC8: modos 1,2,3; 1000 por modo. Total mínimo: 7000 quadros, aproximadamente 11 min 40 s com período nominal de 100 ms, fora configuração e pilotos. Baseline CRC extra pode ser útil, mas não é uma oitava rodada obrigatória pelo PDF.

Mantenha payload, formato, ritmo, domínio de sorteio e circuito. A distribuição dos bits originalmente 1 no campo CHECK pode diferir entre técnicas; por isso reporte tentativas sem mudança e alterações reais, sem presumir denominadores iguais.

Caso real não detectado

Buscar primeiro em SUM8 modo 2. Se não ocorrer em 1000, continuar em rodadas extras identificadas, preservando a tabela original. Arquivar TX/RX hex, seq, posições, tempo, antes/depois e cálculo manual do caso.

Um teste dirigido apagando dois MSBs pode ser apresentado como 'padrão construído' separado da amostra aleatória; não mudar os logs para fingir que foi aleatório. O script de ruído atual não tem comando de ataque dirigido.

Demo extra de 5 a 8 minutos

Mostrar circuito e três papéis; baseline; modo 1 e descarte; tentativas sobre zero; modo 2 e caso previamente registrado; comparação com CRC; timeout/recuperação testado separadamente; previsão dos dez antes de gerar. Essa extensão tem duração própria e não foi embutida na defesa teórica de 20 minutos.

Encerrar com evidências

Dois conjuntos de tabelas, logs brutos, CSVs reconciliados, capturas de bit/intervalo, tempos TX/RX, recuperação em ms e versão de firmware compilada. A prova de 'mudou de fato' e a concordância entre bytes observados, RX e captura, não apenas um GPIO acionado.

Montagem BC547 e primeira validacao - 7 de outubro de 2026

Este documento orienta a montagem que o estudante iniciou nas fotos. As posicoes abaixo sao a distribuicao proposta; as placas ainda nao foram gravadas nem a continuidade eletrica confirmada. A bancada permanece EXTRA.

Identificar as placas

Na segunda foto, olhando como ela foi enviada: superior esquerda = TX; superior direita = RX; inferior = RUIDO. Coloque etiquetas nas placas e anote a porta COM de cada uma. Use os nomes impressos dos GPIOs; a placa inferior esta invertida na foto, por isso nao copie posicoes direita/esquerda entre placas.

Componentes

- BC547 NPN: confirmar a marcacao completa/fabricante. No BC547 TO-92 usual, face plana com texto de frente e pernas para baixo: coletor, base, emissor da esquerda para a direita. O datasheet do componente real prevalece.
- R1 = 1 kohm em serie com o sinal TX->RX.
- R2 = 1 kohm entre GPIO27 da placa RUIDO e a base do BC547.
- R3 = 10 kohm entre base e emissor, recomendado para manter desligado enquanto o GPIO nao e controlado. Se nao houver R3, a condicao de reset deve ser verificada antes de habilitar o injetor.
- GND comum entre as tres placas; cada placa alimentada pelo seu USB. Nao interligar os pinos VIN dos tres USBs.

Dois resistores aparecem nas fotos, mas o estudante informou '1 kohm' sem confirmar separadamente os dois: medir/identificar R1 e R2 antes de considerar o inventario completo.

Circuito e pontos de observacao

```

TX GPIO17 --- A --- R1 1k --- B --- RX GPIO16
      |                   |
      + RUIDO GPIO16 + RUIDO GPIO26
      + RUIDO GPIO25 + coletor BC547

RUIDO GPIO27 --- R2 1k --- base BC547
                   |
                   R3 10k (recomendado)
                   |
GND TX = GND RX = GND RUIDO = emissor BC547
  
```

A = sinal original, antes de R1. B = sinal perturbado, depois de R1. Nenhum jumper pode unir A e B desviando R1. R1 limita a corrente quando o transistor puxa B para GND. O transistor so tenta alterar 1 para 0.

Placa	Pino impresso	Ligar em
TX	GPIO17 / TX2	A
RX	GPIO16 / RX2	B
RUIDO	GPIO16 / RX2	A

Placa	Pino impresso	Ligar em
RUIDO	GPIO25 / D25	A
RUIDO	GPIO26 / D26	B
RUIDO	GPIO27 / D27	R2 -> base
Todas	GND	GND comum e emissor

GPIO16 e GPIO25 da placa RUIDO observam o mesmo ponto A, com funções diferentes. GPIO26 observa B. Isso exige quatro GPIOs mais GND na placa RUIDO; as fotos anteriores não mostram com clareza todas essas conexões.

GPIO18 e GPIO19 da RUIDO ficam reservados como TX das UARTs de observação e devem ficar sem fios; o injetor não transmite por essas UARTs. Isso evita depender da seleção automática de pinos TX pelo pacote Arduino.

O retorno RX GPIO17 -> TX GPIO16 não é usado pelos programas. Se o professor exigir os dois fios cruzados no desenho, pode ser ligado, mas não há ACK nem retransmissão neste protocolo.

Conferir sem alimentação

1. Tirar os cabos USB. Conferir C/B/E pelo fabricante e manter as três pernas em grupos elétricos distintos da protoboard. Em cada lado do canal central, os cinco furos da mesma fileira normalmente são conectados entre si.
2. Confirmar continuidade entre GND das três placas e emissor.
3. Confirmar continuidade entre TX17 e A, A e RUIDO16/25; B e RX16/RUIDO26/coletor.
4. Confirmar R1 de 1 kohm entre A e B sem fio que o desvie. Medir resistores preferencialmente com uma perna fora do circuito, pois caminhos paralelos podem alterar a leitura.
5. Confirmar R2 de 1 kohm entre RUIDO27 e base; não ligar GPIO27 diretamente a A/B.
6. Confirmar ausência de curto de montagem entre alimentação e GND. Cabos USB separados compartilham somente GND/sinais previstos.

Etapa 1 - TX/RX sem transistor conectado a B

Comece com o coletor desligado de B e sem necessidade de ligar a placa RUIDO. Mantenha R1, TX17->A->R1->B->RX16 e GND comum.

Gravar `firmware/tx/tx.ino` na TX e `firmware/rx/rx.ino` na RX com a biblioteca `libraries/Telemetry`. Monitor USB de ambas a 115200 baud.

No RX, enviar r, depois s, depois 0. No TX, enviar s, depois t para vinte quadros. Os comandos são letras/dígitos ASCII minúsculos; podem ser enviados um por vez.

Resultado esperado: vinte registros RX, seq 0..19, residuo 0, ACEITO, payload_confere sim, status OK. O TX envia dados artificiais; não há sensor nesta versão.

Para o primeiro quadro (seq=0), a referência teórica é AA 55 05 00 00 80 80 FB. O piloto precisa confirmar os bytes reais no log; essa referência não é uma medicação.

Se não houver resultado: conferir placa/porta/sketch, GND, TX17/RX16, técnica igual em ambas e biblioteca. Evento AUSÊNCIA antes de iniciar o TX e esperado: não prova falha na fiação.

Etapa 2 - injetor conectado, modo 0

Gravar firmware/noise/noise.ino na RUIDO. Ligar os quatro GPIOs e GND conforme a tabela. Com a placa inicializada, enviar 0 no monitor da RUIDO antes de conectar o coletor a B. Confirmar GPIO27 em LOW/desligado.

Conectar o coletor a B, repetir RX r, s, 0 e TX s, t. Esperado: vinte OK no RX e vinte registros BASELINE na RUIDO com as mesmas sequencias. Se conectar o circuito introduzir erros no modo 0, interromper o piloto e verificar C/B/E, resistor serie, shorts e desligamento do transistor.

Etapa 3 - modos de ruido

Somente depois de baseline estavel, configurar RX r, s, 1, RUIDO 1, TX s, t. Repetir para 2 e 3 em arquivos separados. O digito no RX e metadado, nao aciona ruido; a placa RUIDO precisa receber seu proprio comando.

Nao exigir que toda tentativa produza CORROMPIDO: forcar um zero para zero nao altera o dado. Cruzar o log do injetor com o RX. SEM_PULSO, TIMING_INVALIDO, CAPTURA_INCOMPLETA e CAPTURA_INVALIDA exigem corrigir a tentativa/piloto; nao considerar evidencia de injecao valida.

Capturar A, B e GPIO27 com analisador logico/osciloscopio para verificar pulsos, start/stop, referencia e latencia. Compilacao e logs por si so nao comprovam posicao eletrica.

Trocar para CRC em rodada separada: RX r, c, modo; TX c, t; manter o mesmo modo na RUIDO. Para as rodadas completas, substituir t por g (1000 quadros).

Situacao da validacao

O mapa esta conferido contra o codigo. A continuidade, o funcionamento nas placas, os pulsos e os resultados reais precisam ser confirmados pela equipe. Registrar observacoes sem antecipar que a foto comprova cada ligacao.

Fonte da pinagem: [BC547 - onsemi](#). APIs: [UART Espressif](#) e [GPTimer Espressif](#).

Registro de validacao do codigo - 7 de outubro de 2026

Resultado no computador

Os tres sketches compilaram sem erros para ESP32 classico (esp32:esp32:esp32), com a biblioteca Telemetry deste repositorio. Compilar nao confirma conexoes, funcionamento em placa ou ruido eletrico real.

Sketch	Flash usada (bytes)	Globais em RAM (bytes)
TX	269916	22140
RX	271376	22172
RUIDO	281664	22732

Ferramentas: Arduino CLI 1.5.1 (commit 01f3d4f2b), pacote oficial Espressif Arduino-ESP32 3.3.12. Limites reportados pela configuracao: flash de programa 1310720 bytes e RAM 327680 bytes. Esses valores pertencem a esta compilacao/configuracao, nao medem consumo dinamico durante a execucao.

Nove testes Python passaram, incluindo CRC-8/SMBUS com vetor 123456789 -> F4, erros de um bit, rajadas ate oito bits, colisao SUM8, classes exclusivas e sequencia. O teste C++ do nucleo tambem passou. Cinco cenarios do injetor passaram usando o sketch real com dubles de UART, relógio e GPTimer; instrucoes em tests/README.md.

Correcoes desta revisao

- TX recebeu t para piloto de vinte quadros; g continua com mil.
- TX/RX usam a instancia Serial2 do pacote Arduino; as UARTs do injetor usam Serial1/Serial2, com saidas TX explicitamente reservadas em GPIO18/19, sem conexao.
- Captura parcial do injetor expira pelo ultimo byte observado. Antes, o baseline podia descartar bytes usando o instante antigo de armamento, inclusive antes de receber o cabecalho completo.
- Um alarme atrasado mais de 3 us desliga o BC547 e aborta a tentativa. Falha de rearmamento tambem desliga a saida. Isso reduz uma fonte de pulsos indevidos; nao prova a latencia real da ISR de inicio.
- Cabecalho, sequencia e referencia devem corresponder nos pontos A/B antes de comparar bits. Capturas desemparelhadas geram CAPTURA_INVALIDA, sem fingir evidencia de alteracao.
- Documentacao identifica BC547, resistores com funcoes separadas, mapa GPIO e piloto em tres etapas. A distribuicao de papeis das placas e proposta, pois o estudante ainda nao as havia programado.

Reproduzir

Instalar Arduino CLI e o pacote Espressif 3.3.12, conforme README, e executar na raiz:

```
powershell -ExecutionPolicy Bypass -File tools/compiler.ps1
```

O script tambem aceita -CliPath e -ConfigFile para instalacoes isoladas. Nesta pasta local, detecta automaticamente a instalacao em tools/vendor/; ela nao e publicada no GitHub. Logs e binarios ficam em build/, tambem fora do repositorio. A compilacao nao grava nem abre a porta serial das placas.

Evidências ainda pendentes

Não foram gravadas placas nem coletados dados de bancada nesta verificação. As fotos não confirmam continuidade, pinagem exata C/B/E, valores dos dois resistores, HIGH/LOW, posicionamento dos pulsos, referência intacta, tempos ou recuperação. Não há dados reais em data/reais/.

O próximo passo é o piloto de docs/montagem_bc547.md: primeiro vinte quadros TX/RX sem coletor conectado; depois vinte com o injetor em modo 0; só então testar modos 1..3 e medir A/B/GPIO27. A demonstração física permanece EXTRA.

Fontes e verificacao

Curso Completo de 21 Dias - Telemetria UART com ESP32, fornecido pelo estudante. Consultado integralmente; roteiro vinculado aos dias 1..21.

Trabalho Pratico: Deteccao de Erro em Telemetria sob Ruido Real (ESP32), enunciado fornecido, paginas 1 e 2.

[UART e Arduino-ESP32 - Espressif](#)

[GPTimer - Espressif](#)

[esp_timer - Espressif](#)

Verificacoes de codigo, ferramentas, versoes e limites: consultar a secao Registro de validacao do codigo deste guia.

Nao verificado: funcionamento eletrico, pulsos reais, tempos TX/RX e recuperacao em placa. Nenhuma tabela real foi preenchida.

[BC547 - onsemi](#)

O material teve apoio de IA. A regra do enunciado sobre trabalhos baseados em IA precisa ser observada pela equipe: o relatorio final, as contribuicoes e a defesa precisam refletir o aprendizado e a execucao dos estudantes.